Deploying Angular with Amazon S3 & CloudFront

Hello World Wealth | Student setup guide | Client and Reporting Angular applications

Goal: build an Angular app, store the production files in a private S3 bucket, and serve the site securely over HTTPS using CloudFront. Repeat the workflow for each team and each Angular application.

1. Deployment overview

Browser → CloudFront (HTTPS) → private S3 bucket (Angular HTML, CSS, JavaScript). The Angular code runs in the browser and calls backend APIs separately.

Use distinct resource names for each team and application. S3 bucket names must be globally unique; substitute your own names wherever examples appear.

Application	Example S3 bucket	CloudFront distribution name
Client	hello-world-client-team1-unique	hello-world-client-team1
Reporting	hello-world-reporting-team1-unique	hello-world-reporting-team1

2. Build Angular for production

1. Open Terminal in the Angular project folder.
2. Install dependencies: npm install
3. Create a production build: npx ng build --configuration production
4. Locate the generated index.html. Newer Angular projects often put it in dist/<project-name>/browser/.

IMPORTANT: Upload the contents of the directory containing index.html, not the browser directory itself. index.html must appear at the root of the S3 bucket.

Correct S3 layout: index.html, main-*.js, styles-*.css, assets/ (if present). Incorrect: browser/index.html when the CloudFront origin path is blank.

3. Create the private S3 bucket

5. Open AWS Console → S3 → Create bucket.
6. Choose a globally unique bucket name and the intended region (for this lab, us-east-2 / Ohio is fine).
7. Keep Block all public access ON. Leave static website hosting OFF when using the S3 REST origin with CloudFront OAC.
8. Create the bucket. Open Objects → Upload, and upload all files and subfolders INSIDE the Angular browser/build output directory.
9. Verify index.html is visible directly in the bucket's Objects list.

4. Create the CloudFront distribution

10. Open CloudFront → Distributions → Create distribution. Select Single website configuration if prompted.
11. Enter a descriptive distribution name (for example, hello-world-client-team1). This is not the website URL.
12. For Origin domain, choose the S3 bucket's regional S3 origin (not an S3 website endpoint).
13. Keep Origin path blank when index.html is at the S3 bucket root.
14. Use Origin Access Control (OAC) and enable signing requests. Follow the console instructions to grant the distribution access to the bucket.
15. Set Default root object to index.html (without a leading slash). If another field explicitly requires a path starting with /, that is a different field.
16. Choose the desired CloudFront pricing option and review the current allowances and charges. Create the distribution.

CloudFront assigns a URL such as https://dxxxxxxxxxxxxx.cloudfront.net. A custom domain is optional.

5. Grant CloudFront access to S3

Go to S3 → your bucket → Permissions → Bucket policy. When using OAC, the policy should grant cloudfront.amazonaws.com permission to GetObject only for your CloudFront distribution ARN. Replace ALL placeholders below with your own values:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AllowCloudFrontRead",
    "Effect": "Allow",
    "Principal": {"Service": "cloudfront.amazonaws.com"},
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/*",
    "Condition": {"StringEquals": {
      "AWS:SourceArn": "arn:aws:cloudfront::YOUR_ACCOUNT_ID:distribution/YOUR_DISTRIBUTION_ID"
    }}
  }]
}
```

Keep S3 Block Public Access enabled. Do not add a public-read bucket policy or expose the bucket to everyone.

6. Verify the deployment

17. In CloudFront → Distributions, confirm the distribution is enabled and configuration deployment has completed.

18. Copy the Distribution domain name and open https://YOUR_DOMAIN.cloudfront.net/ in a browser.

19. Also test https://YOUR_DOMAIN.cloudfront.net/index.html to distinguish a default-root-object issue from an origin access issue.

20. Open browser developer tools → Network to verify JavaScript, CSS, and assets load without errors.

21. For Angular client-side routes, configure an SPA fallback carefully: map appropriate 403/404 responses to /index.html with response code 200; note that this may mask genuine missing-asset errors.

7. Troubleshooting

Symptom	What to check
AccessDenied XML / HTTP 403	Confirm index.html at bucket root; Origin path blank; OAC enabled; bucket policy bucket name, account ID and distribution ID match.
Root URL fails; /index.html works	Default root object must be index.html (no leading slash).
Old error after correction	Wait for distribution changes to deploy; create a CloudFront invalidation for /* if necessary.
Blank page / JS 403 or 404	Check S3 build output, asset paths and browser developer tools.
Angular route refresh fails	Configure an appropriate SPA fallback to /index.html.
API request blocked	Check HTTPS mixed content, CORS origin, authentication/JWT, and backend reachability.

8. Updating an Angular deployment

22. Run a new Angular production build.

23. Upload the new build contents to the same S3 bucket root, replacing changed files.

24. Invalidate CloudFront paths as needed (/* is simplest for a lab); wait for invalidation completion.

25. Refresh the browser and confirm the new version. Repeat separately for the Reporting app.

□ Backend API URL, HTTPS, CORS and JWT are addressed before end-to-end testing.